

ASSIGNMENT 2

Web-Based Text Encryption Tool



Name: Ali Waqar

Student ID: 03-134222-055

BSCS-8A

Course: Information Security

1. Project Overview

This report presents a complete solution for the Web-Based Text Encryption Tool assignment. The application is a fully functional single-page web application (SPA) built using plain HTML, CSS, and JavaScript — no backend or installation required. All encryption and decryption logic runs client-side in the browser using native JavaScript APIs and the CryptoJS library loaded from a CDN.

1.1 Core Requirements Met

Requirement	Status	Implementation
Text input box	Done	<textarea> element for plain/cipher text
Algorithm dropdown	Done	<select> with Caesar, Base64, AES-256 options
Encrypt button	Done	Calls process('encrypt') on click
Ciphertext display	Done	Read-only <textarea> shows output
3+ Algorithms	Done	Caesar Cipher, Base64, AES-256
Input Validation	Done	Checks empty text, no algo selected, missing AES key
Decryption (Bonus)	Done	Separate Decrypt button for all reversible algorithms
SHA-256 Hash (Bonus)	Done	Dedicated Hash button using CryptoJS.SHA256()
Copy to Clipboard (Bonus)	Done	Copy button with visual confirmation feedback

2. Encryption Algorithms — Theory & Implementation

2.1 Caesar Cipher

The Caesar Cipher is a classical substitution cipher where each letter in the plaintext is shifted a fixed number of positions in the alphabet. Named after Julius Caesar, who reportedly used a shift of 3.

Formula	Encrypt: $C = (P + \text{shift}) \bmod 26$ Decrypt: $P = (C - \text{shift} + 26) \bmod 26$
---------	--

Example with shift 13 (ROT13): 'Hello World' --> 'Uryyb Jbeyq'

Security Note: Caesar Cipher is NOT secure. It has only 25 possible keys and can be broken in seconds by brute force. It is included here for educational purposes only.

2.2 Base64 Encoding

Base64 is an encoding scheme (not true encryption) that converts binary or text data into a set of 64 printable ASCII characters. It is widely used in email attachments, data URIs, and API payloads.

How it works

Groups input into 3-byte (24-bit) chunks, splits each into four 6-bit groups, and maps each to a Base64 character. Output is always ~33% larger than input and ends with = padding.

Example: 'Hello' --> 'SGVsbG8='

Security Note: Base64 is NOT encryption — it is trivially reversible by anyone with a decoder. Never use it to protect sensitive information.

2.3 AES-256 (Advanced Encryption Standard)

AES is the gold standard of symmetric encryption, standardised by NIST (FIPS 197, 2001) and used by governments and enterprises worldwide to protect classified and sensitive data.

Property	Value	Significance
Key size	256 bits	2^{256} possible keys — computationally infeasible to brute force
Block size	128 bits	Processes data in fixed 128-bit blocks
Rounds	14 (for AES-256)	Each round applies SubBytes, ShiftRows, MixColumns, AddRoundKey
Mode (CryptoJS)	CBC (default)	Cipher Block Chaining — each block depends on the previous
Output format	Base64 string	CryptoJS outputs ciphertext as a Base64-encoded string

Example: 'Hello World' + key 'mysecretkey' --> 'U2FsdGVkX1+...' (long Base64 string)

2.4 BONUS — SHA-256 Hashing

SHA-256 is a one-way cryptographic hash function from the SHA-2 family. Unlike encryption, it is irreversible — you cannot recover the original input from the hash output. It is used for password storage, digital signatures, and file integrity checks.

Key Property

Same input ALWAYS produces the same 256-bit (64 hex character) hash. Any 1-character change completely changes the output — this is the avalanche effect.

Example: 'hello' --> '2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824'

3. Complete Source Code

The entire application is contained in a single HTML file. Save the code below as index.html and open it in any web browser — no installation, no server required.

3.1 index.html — Full Source Code

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Text Encryption Tool</title>
  <!-- CryptoJS Library for AES-256 and SHA-256 -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/crypto-js/4.1.1/crypto-
js.min.js"></script>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: linear-gradient(135deg, #0f172a, #1e293b, #0f172a);
      min-height: 100vh;
      display: flex;
      align-items: center;
      justify-content: center;
      padding: 20px;
    }

    .container {
      background: rgba(255, 255, 255, 0.05);
      backdrop-filter: blur(20px);
      -webkit-backdrop-filter: blur(20px);
      border-radius: 20px;
      padding: 40px;
      width: 100%;
      max-width: 720px;
      border: 1px solid rgba(255, 255, 255, 0.1);
      box-shadow: 0 25px 45px rgba(0, 0, 0, 0.3);
    }

    .header {
      text-align: center;
      margin-bottom: 32px;
    }

    .header h1 {
      color: #fff;
      font-size: 2rem;
```

```

        font-weight: 700;
        margin-bottom: 6px;
        letter-spacing: -0.5px;
    }

    .header p {
        color: #aaa;
        font-size: 0.95rem;
    }

    .badge-row {
        display: flex;
        justify-content: center;
        gap: 8px;
        margin-top: 12px;
        flex-wrap: wrap;
    }

    .badge {
        background: rgba(124, 58, 237, 0.3);
        border: 1px solid rgba(124, 58, 237, 0.5);
        color: #c4b5fd;
        font-size: 0.75rem;
        padding: 3px 10px;
        border-radius: 20px;
        font-weight: 500;
    }

    .form-group {
        margin-bottom: 20px;
    }

    label {
        color: #ccc;
        font-size: 0.88rem;
        font-weight: 500;
        display: block;
        margin-bottom: 7px;
        letter-spacing: 0.3px;
    }

    label span.req {
        color: #f87171;
        margin-left: 2px;
    }

    textarea,
    select,
    input[type="text"],
    input[type="password"] {
        width: 100%;
        padding: 12px 16px;
        border-radius: 12px;
        border: 1px solid rgba(255, 255, 255, 0.12);
        background: rgba(255, 255, 255, 0.07);
        color: #fff;
    }

```

```

        font-size: 0.97rem;
        outline: none;
        font-family: inherit;
        transition: border-color 0.2s, box-shadow 0.2s;
    }

    textarea {
        resize: vertical;
    }

    textarea:focus,
    select:focus,
    input[type="text"]:focus,
    input[type="password"]:focus {
        border-color: #7C3AED;
        box-shadow: 0 0 0 3px rgba(124, 58, 237, 0.25);
    }

    select option {
        background: #1a1a2e;
        color: #fff;
    }

    .key-group {
        display: none;
    }

    .key-group.visible {
        display: block;
    }

    .key-input-wrap {
        position: relative;
    }

    .key-input-wrap input {
        padding-right: 48px;
    }

    .toggle-eye {
        position: absolute;
        right: 14px;
        top: 50%;
        transform: translateY(-50%);
        background: none;
        border: none;
        color: #aaa;
        cursor: pointer;
        font-size: 1.1rem;
        padding: 0;
    }

    .toggle-eye:hover {
        color: #fff;
    }

```

```
.error-msg {
  color: #f87171;
  font-size: 0.82rem;
  margin-top: 5px;
  display: none;
}

.error-msg.show {
  display: block;
}

/* — Buttons ————— */
.btn-row {
  display: flex;
  gap: 10px;
  margin-bottom: 24px;
}

.btn {
  flex: 1;
  padding: 13px 10px;
  border: none;
  border-radius: 12px;
  font-size: 0.92rem;
  font-weight: 600;
  cursor: pointer;
  transition: transform 0.18s, filter 0.18s, box-shadow 0.18s;
  letter-spacing: 0.2px;
}

.btn:hover {
  transform: translateY(-2px);
  filter: brightness(1.12);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.3);
}

.btn:active {
  transform: translateY(0);
}

.btn-encrypt {
  background: linear-gradient(135deg, #7C3AED, #4F46E5);
  color: #fff;
}

.btn-decrypt {
  background: linear-gradient(135deg, #059669, #047857);
  color: #fff;
}

.btn-hash {
  background: linear-gradient(135deg, #D97706, #B45309);
  color: #fff;
}

/* — Output ————— */
```

```
.output-wrapper {
  position: relative;
}

#output {
  background: rgba(0, 0, 0, 0.35);
  color: #00ff99;
  font-family: 'Courier New', Courier, monospace;
  font-size: 0.9rem;
  word-break: break-all;
  letter-spacing: 0.3px;
}

#copyBtn {
  position: absolute;
  top: 10px;
  right: 10px;
  padding: 5px 14px;
  font-size: 0.78rem;
  font-weight: 600;
  background: rgba(124, 58, 237, 0.75);
  border: 1px solid rgba(124, 58, 237, 0.5);
  border-radius: 8px;
  color: #fff;
  cursor: pointer;
  transition: background 0.2s;
}

#copyBtn:hover {
  background: rgba(124, 58, 237, 1);
}

/* — Status bar ————— */
#statusBar {
  margin-top: 12px;
  padding: 10px 16px;
  border-radius: 10px;
  font-size: 0.85rem;
  font-weight: 500;
  display: none;
}

#statusBar.success {
  background: rgba(5, 150, 105, 0.2);
  border: 1px solid rgba(5, 150, 105, 0.4);
  color: #34d399;
  display: block;
}

#statusBar.error {
  background: rgba(220, 38, 38, 0.2);
  border: 1px solid rgba(220, 38, 38, 0.4);
  color: #f87171;
  display: block;
}
```



```
/* — Algorithm info card — */
#algoInfo {
  margin-top: 24px;
  padding: 16px 20px;
  border-radius: 12px;
  background: rgba(255, 255, 255, 0.04);
  border: 1px solid rgba(255, 255, 255, 0.08);
  display: none;
}

#algoInfo.visible {
  display: block;
}

#algoInfo h3 {
  color: #c4b5fd;
  font-size: 0.85rem;
  font-weight: 600;
  margin-bottom: 6px;
  text-transform: uppercase;
  letter-spacing: 0.5px;
}

#algoInfo p {
  color: #94a3b8;
  font-size: 0.85rem;
  line-height: 1.6;
}

.divider {
  border: none;
  border-top: 1px solid rgba(255, 255, 255, 0.08);
  margin: 24px 0;
}

@media (max-width: 500px) {
  .btn-row {
    flex-direction: column;
  }

  .container {
    padding: 24px 18px;
  }
}
</style>
</head>

<body>

  <div class="container">

    <!-- Header -->
    <div class="header">
      <h1><img alt="lock icon" data-bbox="255 853 275 868"/> Text Encryption Tool</h1>
      <p>Encrypt, decrypt, and hash your text securely in the browser</p>
      <div class="badge-row">
```

```

        <span class="badge">Caesar Cipher</span>
        <span class="badge">Base64</span>
        <span class="badge">AES-256</span>
        <span class="badge">SHA-256</span>
    </div>
</div>

<hr class="divider">

<!-- Input Text -->
<div class="form-group">
    <label>Plain Text / Ciphertext <span class="req">*</span></label>
    <textarea id="inputText" rows="4" placeholder="Type or paste your text
here..."></textarea>
    <div class="error-msg" id="textError">⚠ Please enter some text before
proceeding.</div>
</div>

<!-- Algorithm Selector -->
<div class="form-group">
    <label>Encryption Algorithm <span class="req">*</span></label>
    <select id="algorithm" onchange="onAlgoChange()">
        <option value="">— Select an Algorithm —</option>
        <option value="caesar">Caesar Cipher (ROT13 – shift 13)</option>
        <option value="base64">Base64 Encoding / Decoding</option>
        <option value="aes">AES-256 Encryption (Requires Secret Key)</option>
    </select>
    <div class="error-msg" id="algoError">⚠ Please select an algorithm.</div>
</div>

<!-- AES Key (shown only when AES is selected) -->
<div class="key-group form-group" id="keyGroup">
    <label>Secret Key (AES-256) <span class="req">*</span></label>
    <div class="key-input-wrap">
        <input type="password" id="aesKey" placeholder="Enter a strong secret
key...">
        <button class="toggle-eye" onclick="toggleKeyVisibility()"
title="Show/hide key">👁</button>
    </div>
    <div class="error-msg" id="keyError">⚠ AES-256 requires a secret key.</div>
</div>

<!-- Action Buttons -->
<div class="btn-row">
    <button class="btn btn-encrypt" onclick="process('encrypt')">🔒
Encrypt</button>
    <button class="btn btn-decrypt" onclick="process('decrypt')">🔓
Decrypt</button>
    <button class="btn btn-hash" onclick="process('hash')">💧 SHA-256
Hash</button>
</div>

<!-- Output -->
<div class="form-group">
    <label>Output Result</label>

```

```
        <div class="output-wrapper">
            <button id="copyBtn" onclick="copyOutput()"><img alt="copy icon" data-bbox="618 115 635 130"/> Copy</button>
            <textarea id="output" rows="4" readonly placeholder="Result will appear
here..."></textarea>
        </div>
    </div>

    <!-- Status Bar -->
    <div id="statusBar"></div>

    <!-- Algorithm Info Card -->
    <div id="algoInfo">
        <h3><input type="checkbox"/> About this Algorithm</h3>
        <p id="algoInfoText"></p>
    </div>

</div>

<script>
    // _____
    // Algorithm info descriptions
    // _____
    const algoDescriptions = {
        caesar: "Caesar Cipher shifts each letter by 13 positions (ROT13). It is a
classical substitution cipher used for education only – it has just 25 possible keys
and can be broken instantly. Non-letter characters are left unchanged.",
        base64: "Base64 is an encoding scheme (NOT encryption) that converts text
into printable ASCII characters. It is widely used in emails and APIs. Anyone can
decode it – do not use it to protect sensitive data.",
        aes: "AES-256 is the gold standard of symmetric encryption, used by
governments and banks worldwide. It requires a secret key – the same key used to
encrypt must be used to decrypt. Without the key, the ciphertext is computationally
infeasible to break."
    };

    // _____
    // Show/hide AES key field + algorithm info card
    // _____
    function onAlgoChange() {
        const algo = document.getElementById('algorithm').value;

        // AES key field
        document.getElementById('keyGroup').className =
            'key-group form-group' + (algo === 'aes' ? ' visible' : '');

        // Algorithm info card
        const infoBox = document.getElementById('algoInfo');
        const infoText = document.getElementById('algoInfoText');
        if (algo && algoDescriptions[algo]) {
            infoText.textContent = algoDescriptions[algo];
            infoBox.className = 'visible';
        } else {
            infoBox.className = '';
        }

        clearErrors();
    }
</script>
```

```
    }

    // -----
    // Toggle AES key visibility
    // -----
    function toggleKeyVisibility() {
        const inp = document.getElementById('aesKey');
        inp.type = inp.type === 'password' ? 'text' : 'password';
    }

    // -----
    // Main process function
    // -----
    function process(mode) {
        clearErrors();
        hideStatus();

        const text = document.getElementById('inputText').value.trim();
        const algo = document.getElementById('algorithm').value;
        const key = document.getElementById('aesKey').value.trim();

        // — Validation —————
        let valid = true;

        if (!text) {
            show('textError');
            valid = false;
        }

        if (mode !== 'hash' && !algo) {
            show('algoError');
            valid = false;
        }

        if (algo === 'aes' && !key) {
            show('keyError');
            valid = false;
        }

        if (!valid) return;

        // — Encryption / Decryption / Hashing —————
        let result = '';
        let label = '';

        try {
            if (mode === 'hash') {
                // SHA-256 — always available regardless of selected algo
                result = CryptoJS.SHA256(text).toString();
                label = 'SHA-256 Hash';

            } else if (algo === 'caesar') {
                result = caesarCipher(text, 13, mode === 'decrypt');
                label = mode === 'encrypt' ? 'Caesar Encrypted (ROT13)' : 'Caesar
Decrypted';
            }
        } catch {
            show('error');
        }
    }
}
```

```
    } else if (algo === 'base64') {
      if (mode === 'encrypt') {
        result = btoa(unescape(encodeURIComponent(text)));
        label = 'Base64 Encoded';
      } else {
        result = decodeURIComponent(escape(atob(text)));
        label = 'Base64 Decoded';
      }
    }

    } else if (algo === 'aes') {
      if (mode === 'encrypt') {
        result = CryptoJS.AES.encrypt(text, key).toString();
        label = 'AES-256 Encrypted';
      } else {
        const bytes = CryptoJS.AES.decrypt(text, key);
        result = bytes.toString(CryptoJS.enc.Utf8);
        if (!result) throw new Error('Decryption failed. Wrong key or
corrupted ciphertext.');
```

corrupted ciphertext.');

```
        label = 'AES-256 Decrypted';
      }
    }

    document.getElementById('output').value = result;
    showStatus('success', '✅ ' + label + ' – done successfully!');

  } catch (err) {
    document.getElementById('output').value = '';
    showStatus('error', '❌ Error: ' + err.message);
  }
}

// =====
// Caesar Cipher (ROT-N)
// =====
function caesarCipher(text, shift, decrypt) {
  const s = decrypt ? (26 - shift) % 26 : shift;
  return text.split('').map(ch => {
    if (/[A-Z]/.test(ch)) return String.fromCharCode((ch.charCodeAt(0) - 65
+ s) % 26 + 65);
    if (/[a-z]/.test(ch)) return String.fromCharCode((ch.charCodeAt(0) - 97
+ s) % 26 + 97);
    return ch; // digits, punctuation, spaces unchanged
  }).join('');
}

// =====
// Copy output to clipboard
// =====
function copyOutput() {
  const out = document.getElementById('output');
  if (!out.value) {
    showStatus('error', '❌ Nothing to copy – run an operation first.');
```

Nothing to copy – run an operation first.');

```
    return;
  }
  navigator.clipboard.writeText(out.value).then(() => {
```

```
        const btn = document.getElementById('copyBtn');
        btn.textContent = '✅ Copied!';
        setTimeout(() => { btn.textContent = '📋 Copy'; }, 2000);
    }).catch(() => {
        // Fallback for older browsers
        out.select();
        document.execCommand('copy');
        showStatus('success', '✅ Copied to clipboard!');
    });
}

// -----
// Helpers
// -----

function show(id) { document.getElementById(id).classList.add('show'); }
function clearErrors() { ['textError', 'algoError', 'keyError'].forEach(id =>
document.getElementById(id).classList.remove('show')); }
function hideStatus() { const s = document.getElementById('statusBar');
s.className = ''; s.textContent = ''; }
function showStatus(type, msg) {
    const s = document.getElementById('statusBar');
    s.className = type;
    s.textContent = msg;
}
}
</script>

</body>

</html>
```

4. How to Run the Application

4.1 Local Setup (Zero Installation)

1. Copy the full source code from Section 3 above.
2. Open any text editor (Notepad, VS Code, Sublime Text).
3. Paste the code and save the file as index.html.
4. Double-click the file — it opens in your default browser instantly.
5. No web server, no npm, no Python — it runs entirely in the browser.

4.2 GitHub Pages Deployment

6. Create a new public GitHub repository named “web-encryption-tool”.
7. Upload your index.html file to the repository root.
8. Go to: Settings > Pages > Source > Deploy from branch (main).
9. GitHub generates a live URL: <https://yourusername.github.io/web-encryption-tool>
10. Share this URL for your demo — instructors can access it without downloading anything.

Submission Tip

Upload index.html + a README.md explaining the project. The README should describe each algorithm, how to run, and what the bonus features do.

5. Feature Walkthrough

Feature	Steps	Expected Result
Caesar Encrypt	Enter text > Select Caesar Cipher > Click Encrypt	Shifted letters (Hello -> Uryyb with ROT13)
Caesar Decrypt	Paste ciphertext > Select Caesar Cipher > Click Decrypt	Original plaintext restored
Base64 Encode	Enter text > Select Base64 > Click Encrypt	Base64 string ending with = (e.g. SGVsbG8=)
Base64 Decode	Paste Base64 text > Select Base64 > Click Decrypt	Original text restored
AES Encrypt	Enter text + key > Select AES-256 > Click Encrypt	Long encrypted Base64 string
AES Decrypt	Paste AES output + same key > Click Decrypt	Original plaintext restored
SHA-256 Hash	Enter any text > Click SHA-256 button	64-character hex hash string
Copy Output	Click Copy after any operation	Text copied to clipboard; button says 'Copied!'

6. Input Validation Logic

Three validation checks run before any operation is executed:

Validation	Trigger	Error Message Shown
Empty text	Input textarea is blank	'Please enter some text.'
No algorithm	Dropdown still on default option	'Please select an algorithm.'
Missing AES key	AES selected but key field empty	'AES requires a secret key.'

All errors are displayed in red beneath the relevant field. They are cleared automatically on the next button press. No operation proceeds unless all applicable checks pass.

7. Security Analysis

Algorithm	Type	Security Level	Real-World Use
Caesar Cipher	Substitution cipher	Very Weak — only 25 keys	Education, puzzles only
Base64	Encoding scheme	No security — easily reversed	Data transfer encoding
AES-256	Symmetric block cipher	Very Strong — industry standard	File encryption, HTTPS, banking
SHA-256	Cryptographic hash	Strong — one-way, collision-resistant	Password storage, digital signatures

Key takeaway: For any production application handling real user data, only AES-256 or higher should be used for encryption. Caesar Cipher and Base64 are included here strictly for demonstrating different concepts in an educational context.

8. Bonus Features Implemented

Bonus Item	Status	How Implemented
Decryption feature	Implemented	Separate green Decrypt button; works for Caesar, Base64, AES
SHA-256 hashing	Implemented	Amber Hash button using CryptoJS.SHA256() — output is hex string
Copy to clipboard	Implemented	Copy button uses navigator.clipboard.writeText(); shows 'Copied!' for 2s
User authentication	Not required	Out of scope — listed as optional advanced feature only

9. Conclusion

This assignment delivers a complete, working web-based text encryption tool that satisfies all core requirements and three of the four bonus features. The application demonstrates practical knowledge of classical and modern cryptographic techniques — from the simple Caesar Cipher to industry-standard AES-256 — implemented entirely in client-side JavaScript.

The project is lightweight, portable, and deployable anywhere as a single HTML file. The clean dark-themed UI, real-time validation, and bonus features make it a polished deliverable suitable for both academic submission and real demonstration.

GitHub Repo

Repo: web-encryption-tool | Files: index.html + README.md | Deploy via GitHub Pages for live demo link

10. References

11. NIST. (2001). Advanced Encryption Standard (AES). FIPS PUB 197.
<https://csrc.nist.gov/publications/detail/fips/197/final>
12. RFC 4648 — The Base16, Base32, and Base64 Data Encodings.
<https://datatracker.ietf.org/doc/html/rfc4648>
13. CryptoJS Documentation. <https://cryptojs.gitbook.io/docs>
14. NIST. (2001). Secure Hash Standard (SHA). FIPS PUB 180-4.
<https://csrc.nist.gov/publications/detail/fips/180/4/final>
15. MDN Web Docs. Clipboard API. https://developer.mozilla.org/en-US/docs/Web/API/Clipboard_API